ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI THỰC HÀNH

MÔN HỌC: CE118 - THIẾT KẾ LUẬN LÝ SỐ

Lớp: CE118.Q14.2

LAB 06: THIẾT KẾ BỘ XỬ LÝ MIPS ĐƠN GIẢN

Giảng viên hướng dẫn: **ThS. Hồ Ngọc Diễm**

Sinh viên thực hiện: **Lương Cao Pháp – 24521288**

TP. HỒ CHÍ MINH – Tháng 12 năm 2025

THỰC HÀNH THIẾT KẾ LUẬN LÝ SỐ - LỚP CE118.Q14.2

BÀI THỰC HÀNH 06: THIẾT KẾ BỘ XỬ LÝ MIPS ĐƠN GIẢN

1. Mục tiêu

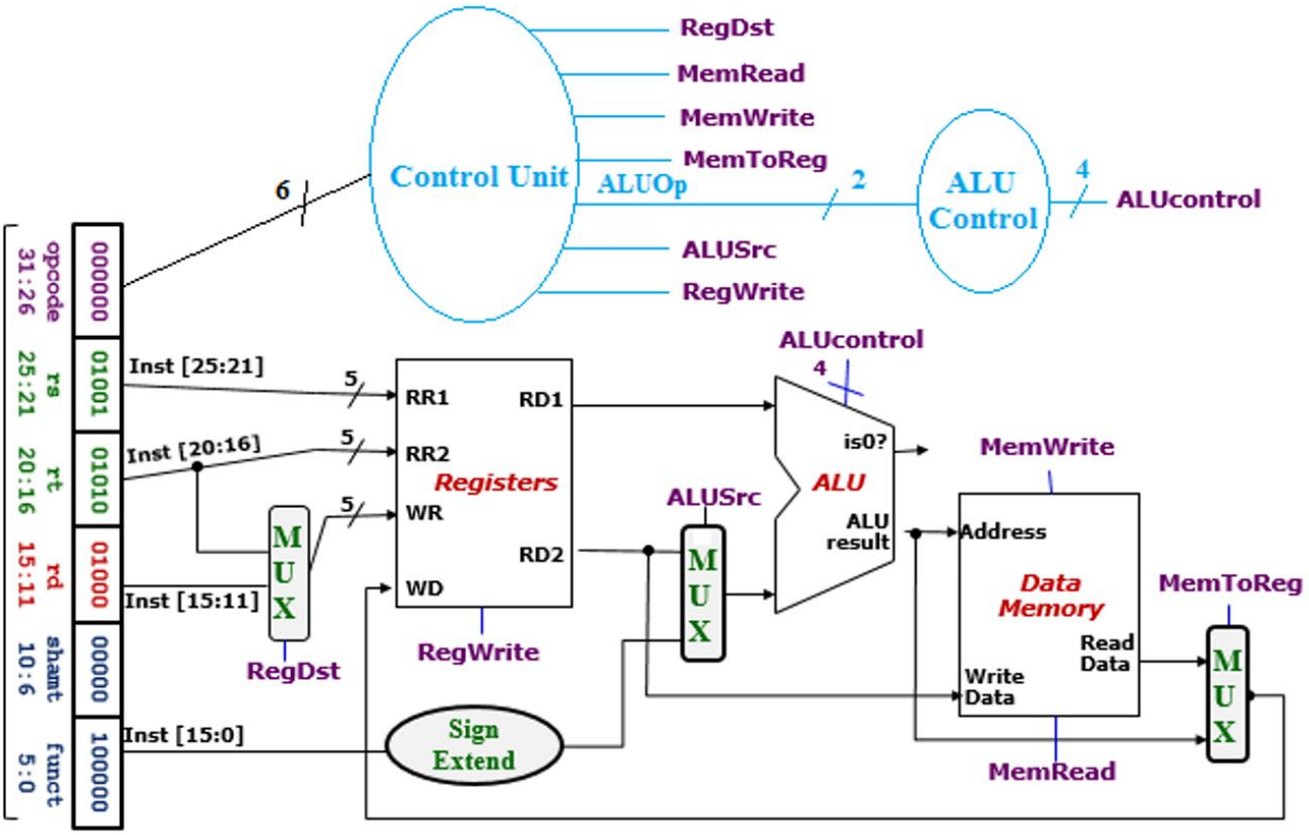
- Sinh viên ôn tập kiến thức về tập lệnh và Datapath của MIPS. Thiết kế Datapath và Control Unit cho bộ xử lý đơn chu kỳ theo kiến trúc MIPS.

2. Nội dung

b. Thực hành trên lớp

Câu 1: Sinh viên dựa vào lý thuyết về DATAPATH của kiến trúc MIPS đã học trong môn Kiến trúc máy tính như trên Hình 1 và sử dụng lại các module đã thiết kế trong các lab trước để thiết kế Datapath và Control Unit thực hiện được một số lệnh cơ bản của MIPS như:

- add $1, $2, $3
- addi $4, $5, -23
- slt $3, $2, $1
- lw $1, 16($2)
- sw $1, 0($2)

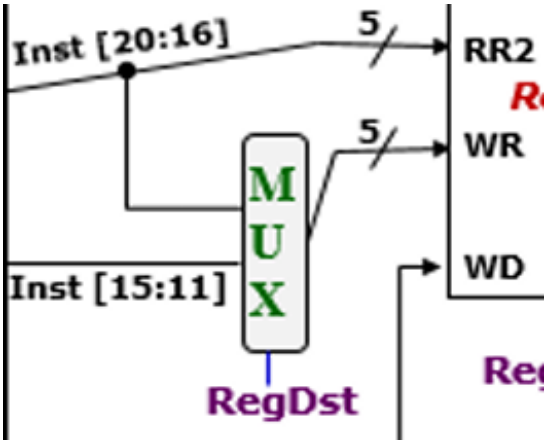


Hình 1. Datapath và Control Unit theo kiến trúc MIPS

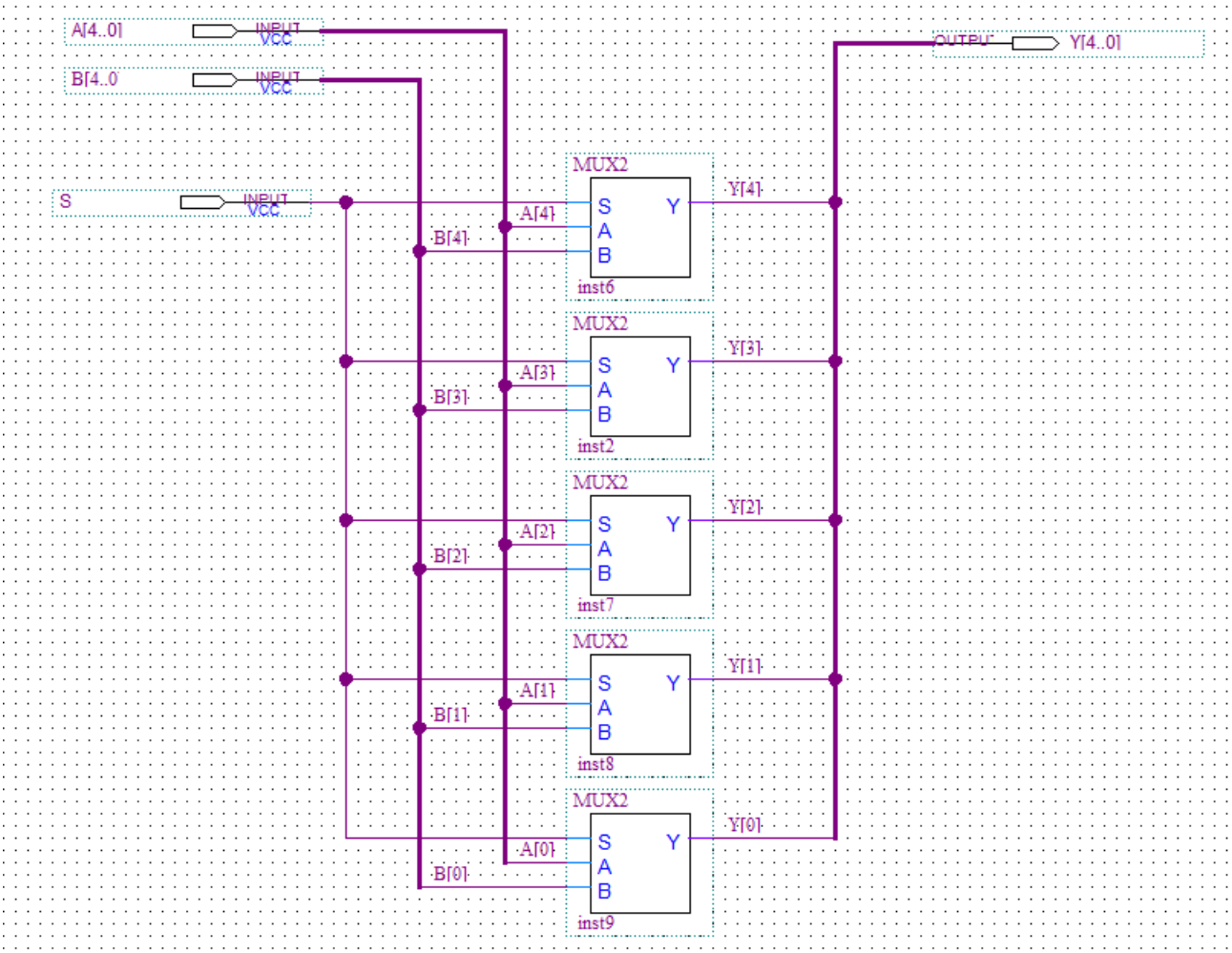
A. THIẾT KẾ CÁC THÀNH PHẦN CỦA DATAPATH

1. Thiết kế mạch MUX 2-1 data 5 bit

- Đối với lệnh R-type và I-type, ta cần chọn thanh ghi đích để ghi kết quả tính toán được vào thanh ghi đó. Vì bit lưu giá trị thanh ghi đích ở 2 loại lệnh này ở vị trí khác nhau trên mã máy nên ta cần sử dụng 1 MUX 2-1 data 5 bit để lựa chọn giá trị thanh ghi đích cần ghi kết quả tính toán vào. Tín hiệu điều khiển MUX chọn thanh ghi đích này được điều khiển bởi tín hiệu output của khối Control tương ứng với loại lệnh này là RegDst.



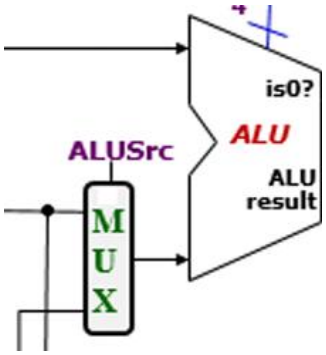
- Vẽ mạch:



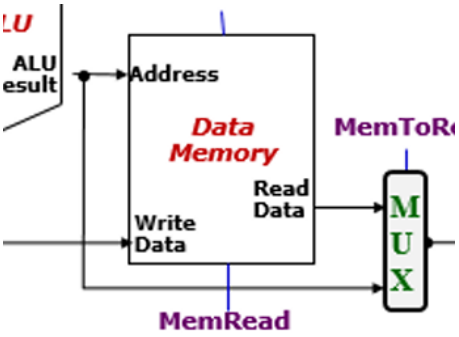
2. Thiết kế mạch MUX 2-1 data 32 bit

- MUX 2-1 data 32 bit được sử dụng để lựa chọn dữ liệu ở 2 vị trí:

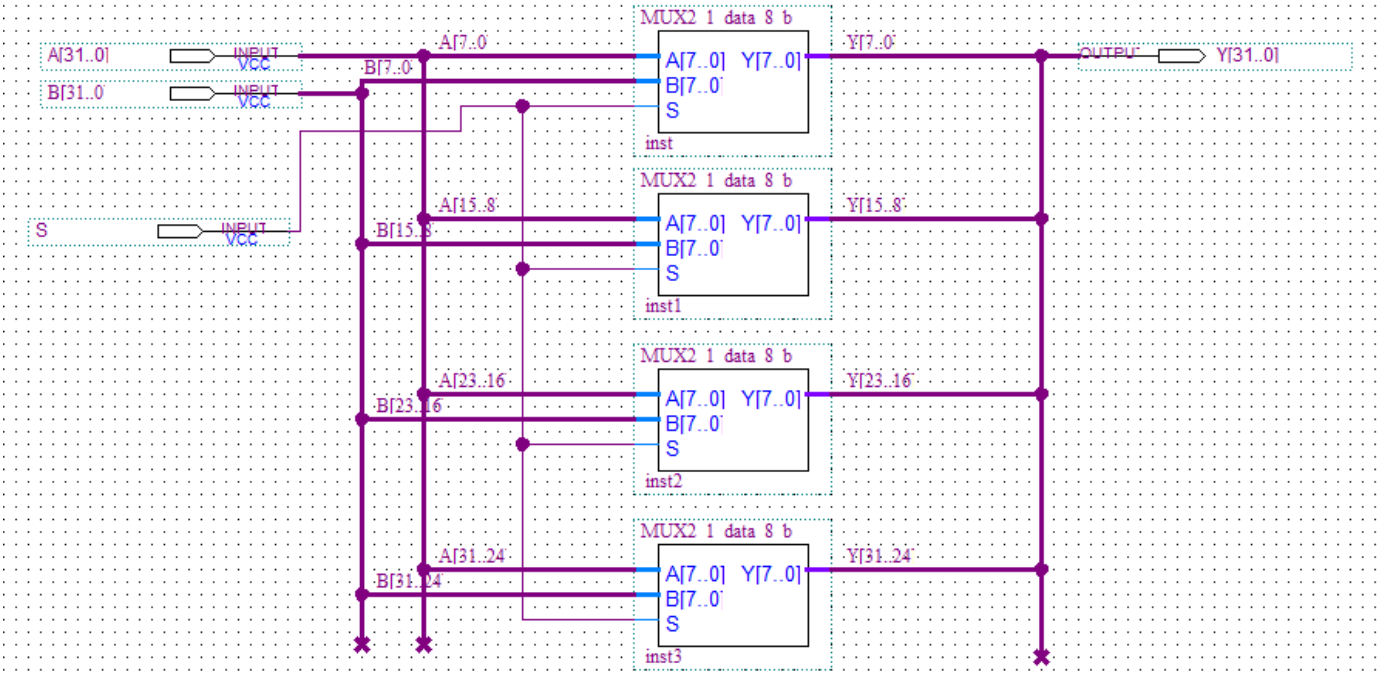
+ Lựa chọn toán tử thứ 2 cho khối ALU từ giá trị Read Data 2 hoặc từ khối Sign Extend bằng tín hiệu ALUSrc:



+ Lựa chọn giá trị ghi vào thanh ghi đích là từ kết quả tính toán của ALU hoặc từ giá trị đọc được từ bộ nhớ Data Memory bằng tín hiệu MemToReg:



- Vẽ mạch:



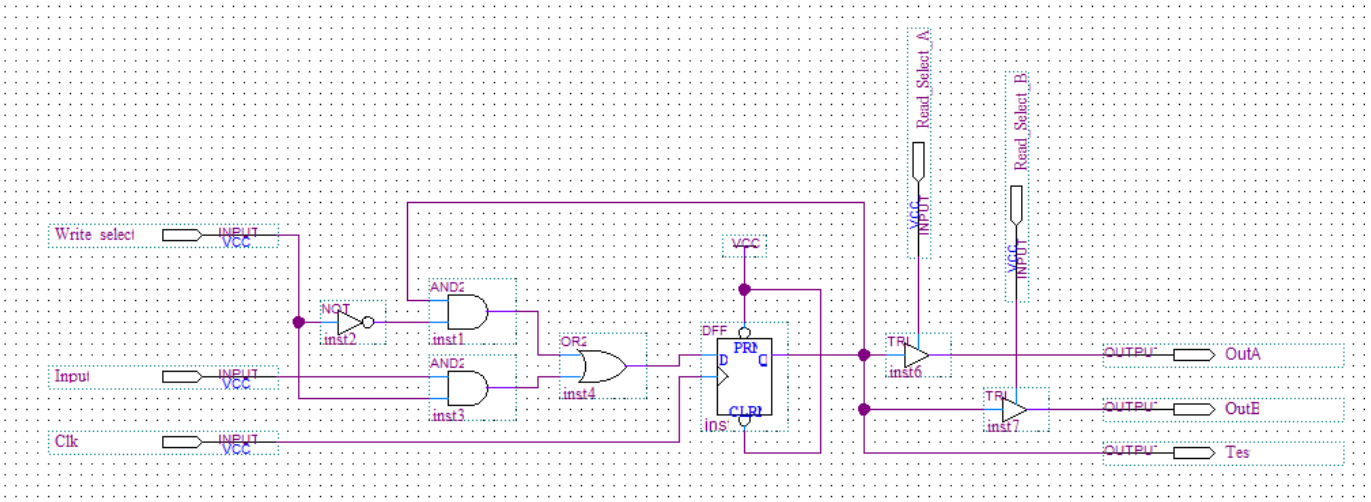
Hình 2. Mạch MUX 2-1 data 32 bit bằng cách ghép 4 MUX 2-1 data 8 bit

3. Thiết kế tập thanh ghi Register File

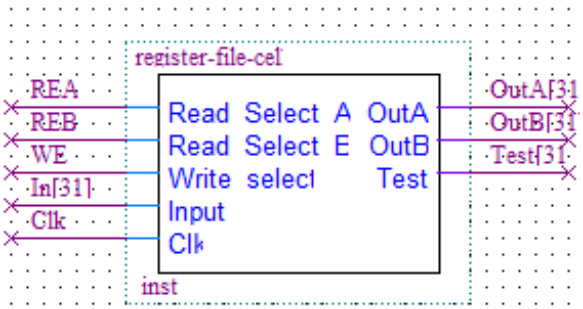
- Tập thanh ghi (Register File) trong MIPS gồm có 32 thanh ghi có kích thước dữ liệu là 32 bit.

- Sử dụng các thanh ghi có 1 port Write và 2 port Read. Các thanh ghi này có tín hiệu cho phép ghi Write Enable (WE) và cho phép đọc Read Enable 1 (RE1) và Read Enable 2 (RE2). Ta sử dụng các bộ Decoder 5 to 32 để mỗi khi truyền địa chỉ của thanh ghi, Decoder sẽ giải mã các địa chỉ này từ đó cho phép các thanh ghi thực hiện thao tác đọc và ghi tương ứng.

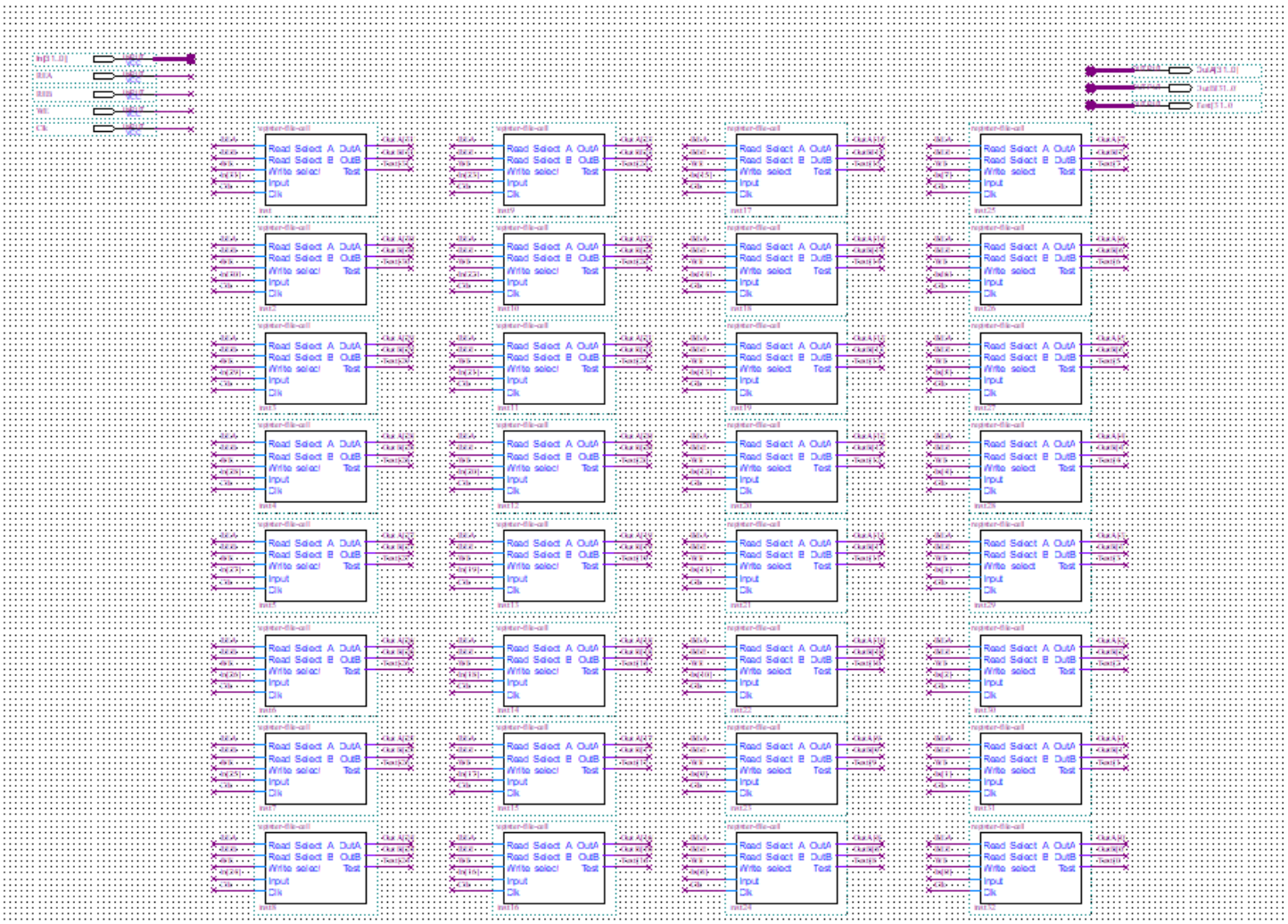
- Thiết kế tế bào thanh ghi Register-Cell có 1 port Write và 2 port Read:



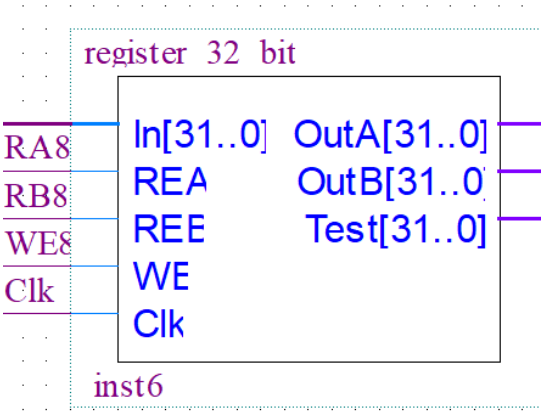
- Đóng gói tế bào thanh ghi Register-Cell:



- Thiết kế thanh ghi 32 bit từ 32 Register-Cell:



- Đóng gói thanh ghi 32 bit:



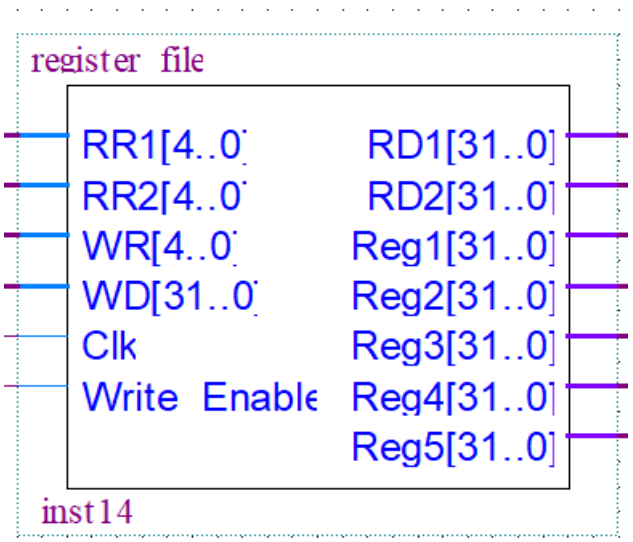
- Thiết kế tập thanh ghi Register File từ 32 thanh ghi:



Tập thanh ghi gồm có:

- 32 thanh ghi 32 bit, trong đó thanh ghi $0 không cho phép nạp giá trị, giá trị luôn luôn là 0, các thanh ghi còn lại đều có 1 port Write và 2 port Read.
- 1 Decoder dùng để giải mã địa chỉ cho phép ghi, 2 Decoder dùng để giải mã địa chỉ cho phép đọc.

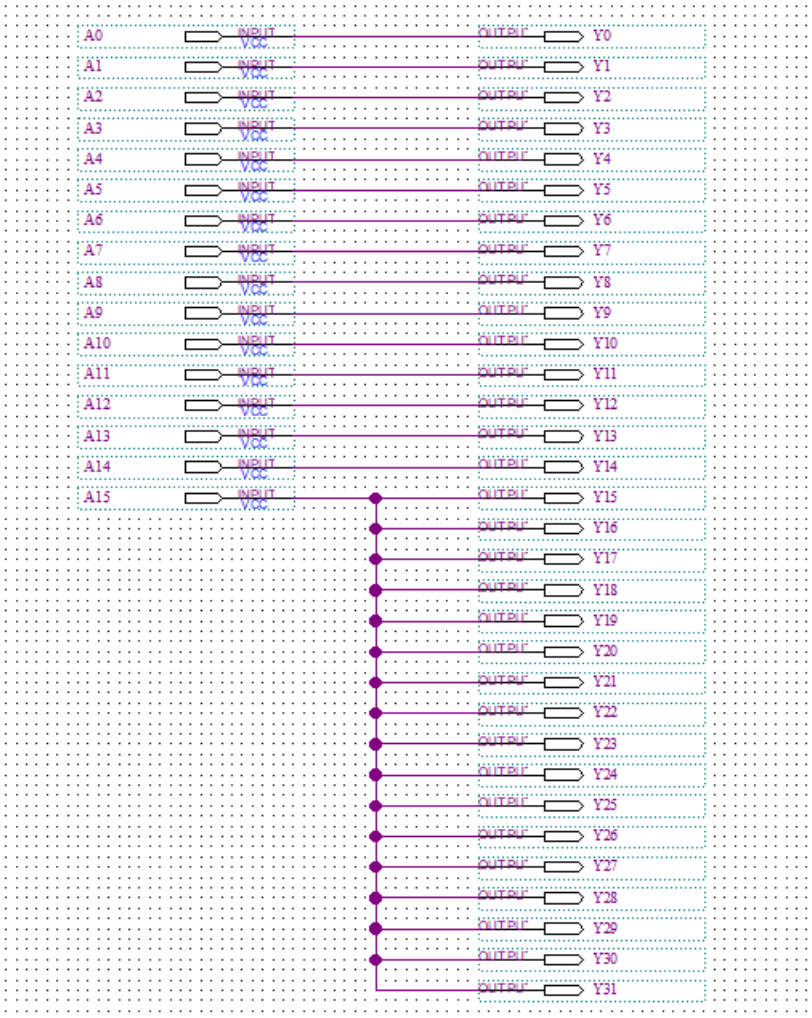
- Đóng gói tập thanh ghi:



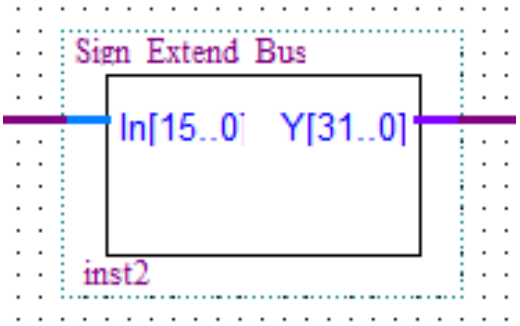
4. Thiết kế khối mở rộng dấu Sign-Extend

- Khối Sign Extend có nhiệm mở rộng giá trị của trường Immediate trong mã máy từ 16 bit thành giá trị 32 bit có dấu. Các bit ở vị trí [31:16] của khối output Sign Extend có giá trị bằng giá trị bit dấu của Immediate.

- Vẽ mạch:



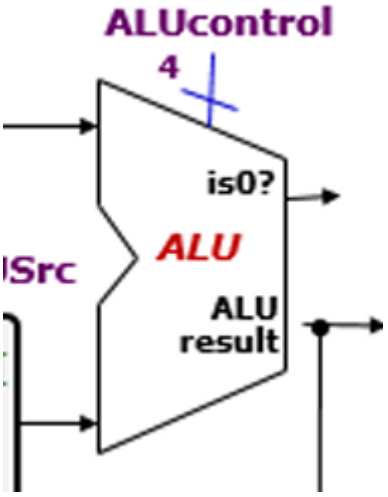
- Đóng gói mạch:



5. Thiết kế khối ALU

- Khối ALU bao gồm khối Arithmetic và Logic Unit có thể hỗ trợ tính toán gồm 6 toán hạng (AND, OR, add, sub, slt and NOR). Ngoài ra, khối ALU còn có thêm 1 cờ báo isZero dùng để kiểm tra xem kết quả của ALU có bằng 0 hay không.

- Các phép toán của ALU được lựa chọn bởi tín hiệu điều khiển ALUControl 4 bit được điều khiển bởi khối Control:



- Bảng sự thật của khối ALU:

Function	ALUControl[3:0]				Result
AND	0	0	0	0	F = A & B, isZero cập nhật
OR	0	0	0	1	F = A | B, isZero cập nhật
add	0	0	1	0	F = A + B, isZero cập nhật
sub	0	1	1	0	F = A - B, isZero cập nhật
slt	0	1	1	1	F = (A < B) ? 1 : 0, isZero cập nhật
NOR	1	1	0	0	F = ~(A | B), isZero cập nhật

- Ý tưởng thiết kế:

- Đối với phép AND và phép OR, ta sử dụng các cổng logic AND và OR để tính toán kết quả.
- Đối với phép toán NOR, ta thấy rằng:

$$A\ NOR\ B = \sim (A\ OR\ B) = \sim A\ AND \sim B$$

Như vậy, bản chất phép NOR là sử dụng lại phép AND đã được thiết kế với các giá trị toán tử truyền vào là NOT A và NOT B.

- Đối với phép toán cộng (add) hai số, ta sử dụng bộ cộng Full Adder cộng các bit của hai số lại với nhau. Đối với phép toán trừ (sub), ta áp dụng phương pháp bù hai (two's complement):

$$A - B = A + (bù\ 2\ của\ B) = A + (bù\ 1\ của\ B) + 1 = A + \sim B + 1$$

Vì vậy ta chỉ cần dùng mạch cộng đã có ở phép toán cộng và chuyển B thành B' sau đó cộng 1 bằng cách sử dụng Carry_In thấp nhất đặt lên giá trị là 1.

- Đối với phép slt (Set Less Than), nếu A < B thì kết quả ALU tính toán là 1. Ta sử dụng lại phép toán trừ A – B. Sau khi đã có kết quả của phép trừ, ta tiến hành kiểm tra bit dấu của kết quả (tức bit kết quả của Full Adder cuối cùng). Nếu bit dấu là 1 (tức A – B là một số âm) thì A < B và kết quả ALU cuối cùng là 1. Ngược lại nếu bit dấu là 0 (tức A – B là 0 hoặc là một số nguyên dương) thì A >= B và kết quả ALU cuối cùng là 0. Tuy nhiên có một vấn đề xảy ra là phép trừ hai số A và B có thể bị tràn (overflow) vì vậy lúc này ta cần ràng buộc thêm điều kiện overflow như sau:

Bit dấu của A - B	**Overflow**	**Kết quả cuối cùng của ALU**
0	0	0
1	0	1
0	1	1
1	1	0

Từ bảng trên ta thu được biểu thức:

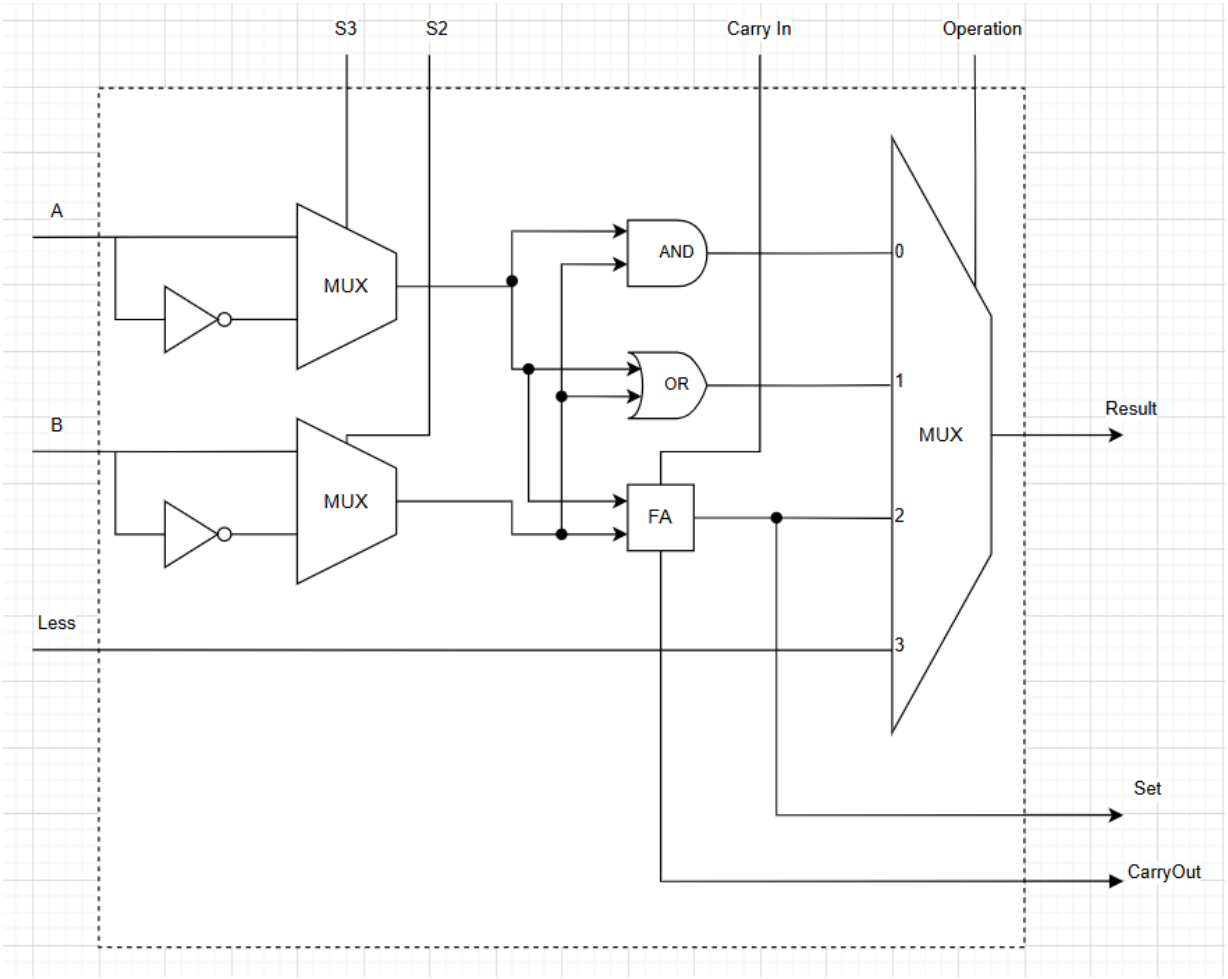
Set = MSB của (A - B) xor Overflow

Từ phân tích các phép toán trên với tín hiệu điều khiển ALUControl[3:0] ta có thể phân loại thành 4 nhóm để lựa chọn kết quả cuối cùng trong khối ALU:

- Tín hiệu XX00: kết quả AND hoặc NOR.
- Tín hiệu XX01: kết quả phép OR
- Tín hiệu XX10: kết quả phép ADD hoặc SUB
- Tín hiệu XX11: kết quả phép SLT

Ngoài ra ta nhận thấy rằng, bit thứ 2 và thứ 3 của tín hiệu ALUControl quyết định việc lấy giá trị của toán tử truyền vào (nếu giá trị bằng 0) hoặc lấy giá trị sau khi NOT giá trị của toán tử truyền vào này (nếu giá trị bằng 1).

- Ta tiến hành thiết kế tế bào của khối ALU cho mỗi bit dữ liệu A và B:

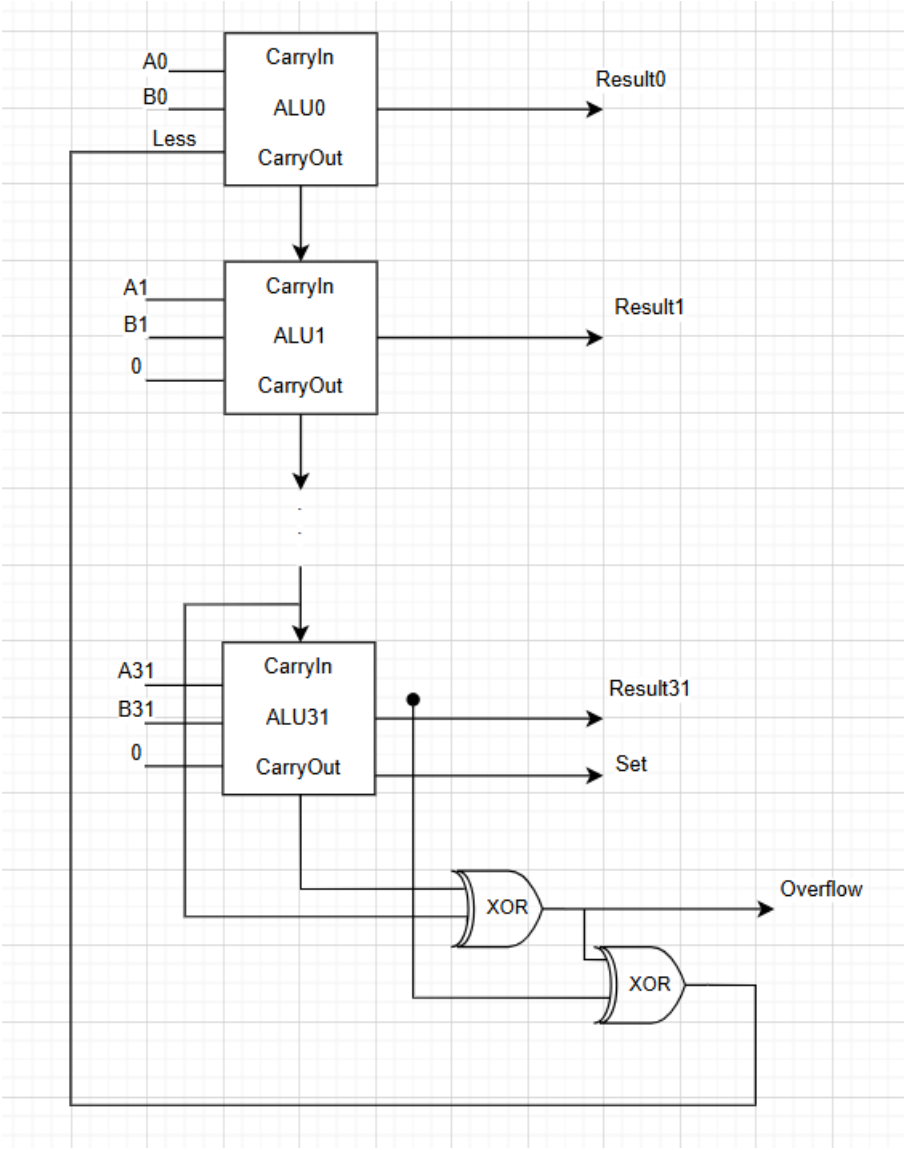


Hình 3. Tế bào ALU 1 bit

Mỗi tế bào ALU 1 bit gồm có:

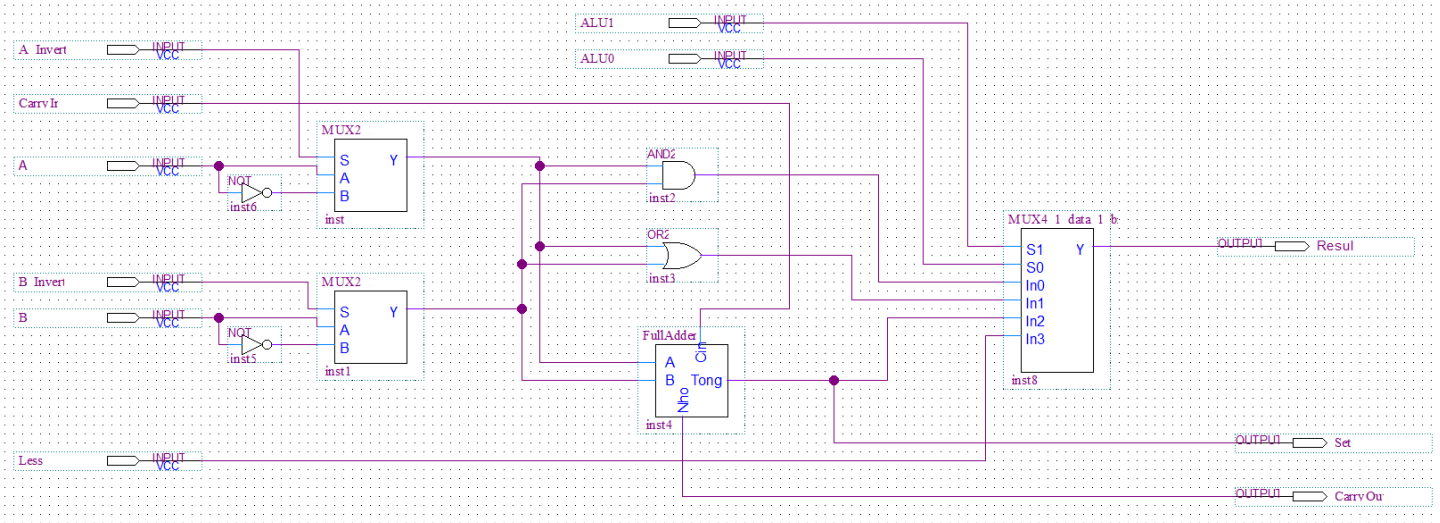
- Các ngõ vào:
  - Bit A và bit B tương ứng
  - Tín hiệu S3, S2 (tức ALUControl[3:2]) dùng để lựa chọn giá trị toán hạng
  - CarryIn là bit nhớ của Full Adder ở phía trước
  - Operation (tức ALUControl[1:0]) dùng để lựa chọn giá trị cuối cùng cho bit kết quả của tế bào ALU này.
- Các ngõ ra:
  - Result là kết quả của bit tương ứng sau khi thực thi phép toán với bit của A và B
  - Set là kết quả tính toán của Full Adder để sau khi tính toán xong phép trừ, ta sẽ sử dụng giá trị Set của tế bào ALU cuối cùng để xét giá trị của phép SLT.
  - CarryOut là bit nhớ dùng để truyền vào Full Adder ở phía sau.

Sau khi có được tế bào ALU 1 bit, ta tiến hành ghép 32 tế bào ALU lại thành khối ALU hoàn chỉnh:

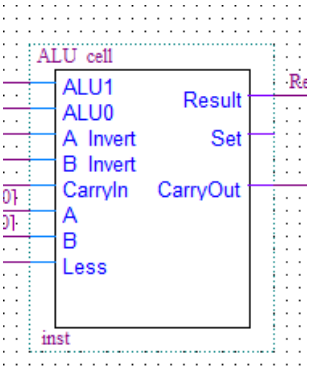


Hình 4. Sơ đồ thiết kế ALU data 32 bit từ 32 tế bào ALU 1 bit

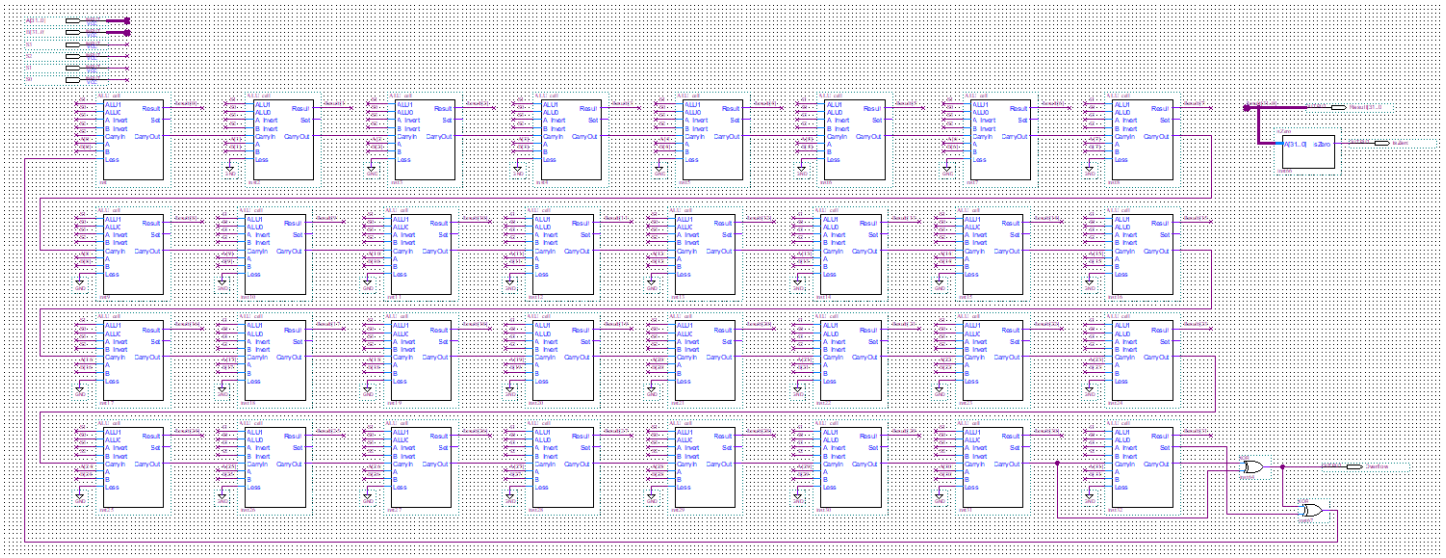
- Vẽ mạch tế bào ALU 1 bit:



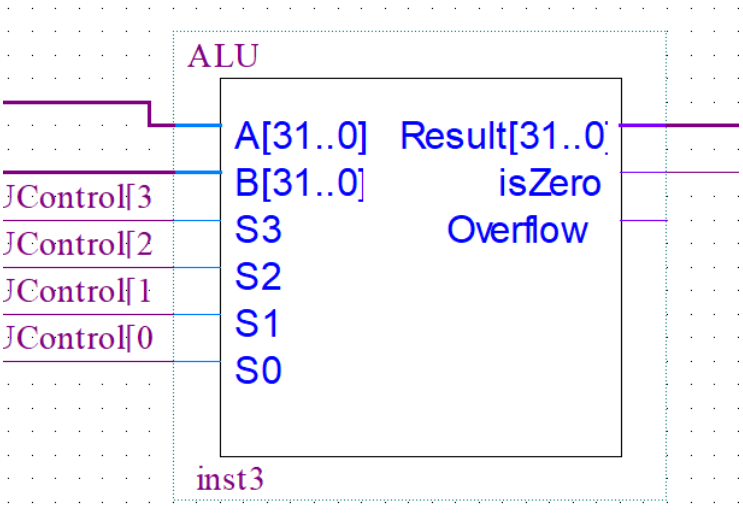
- Đóng gói mạch tế bào ALU 1 bit:



- Vẽ mạch ALU 32 bit:



- Đóng gói mạch ALU 32 bit:



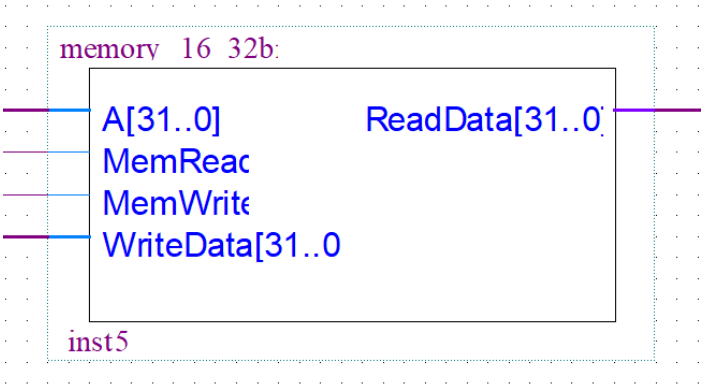
6. Thiết kế khối bộ nhớ Data Memory

- Do chỉ dùng mô phỏng nên ta chỉ thiết kế kích thước Data Memory với kích thước khoảng 16 x 32 bit. Trong bài này ta tái sử dụng lại mạch đã thiết kế là RAM 4x4, ta ghép nối tiếp 8 RAM 4x4 thành RAM 4x32 và ghép 4 RAM 4x32 song song lại thành RAM 16x32 bit.

- Thiết kế RAM 16x32 bit:

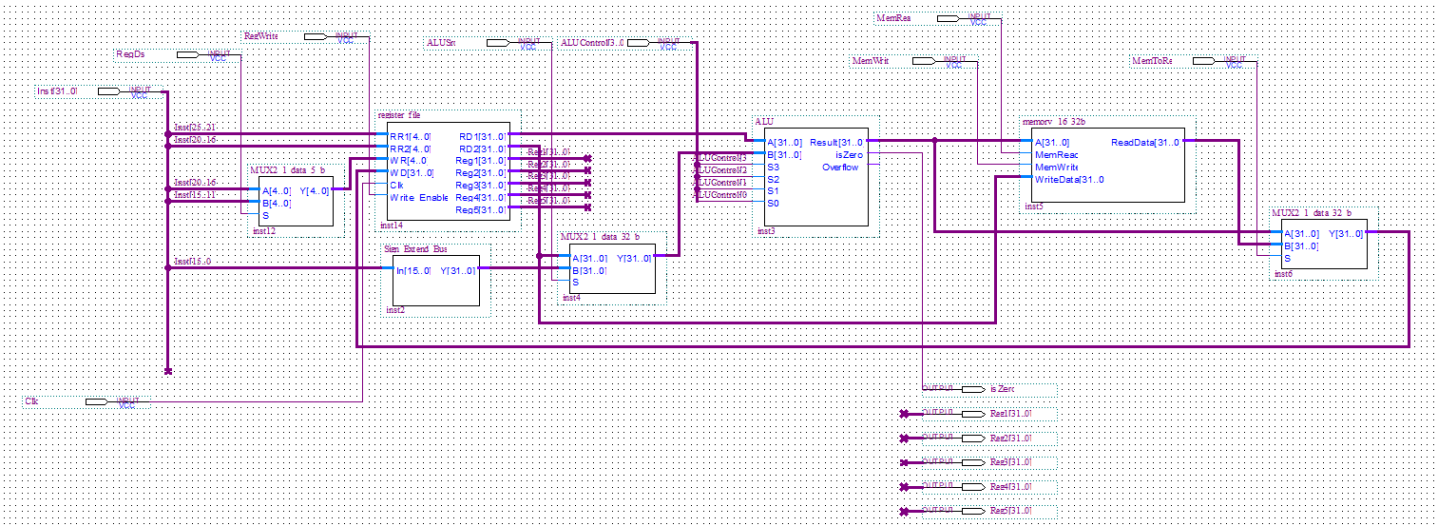


- Đóng gói RAM 16x32 bit:



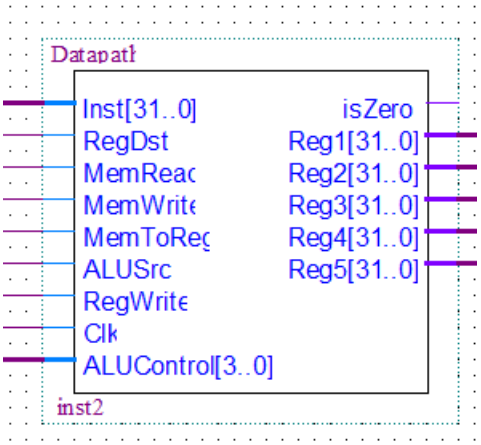
7. Thiết kế khối Datapath

- Sau khi đã thiết kế các thành phần trong khối Datapath, ta tiến hành kết nối các thành phần này lại với nhau thành Datapath hoàn chỉnh:



Ta thêm các output của các thanh ghi $1, $2, $3, $4, $5 để quan sát kết quả mô phỏng.

- Đóng gói Datapath:



B. THIẾT KẾ CÁC THÀNH PHẦN CỦA CONTROL UNIT

1. Thiết kế khối Control Unit

- Khối Control Unit dựa vào giá trị Opcode của mã máy Inst[31:26] dùng để xác định các giá trị điều khiển của các khối trong Datapath. Riêng đối với tín hiệu ALUControl[3:0] dùng để lựa chọn phép toán cho ALU thì Control Unit không xuất tín hiệu điều khiển trực tiếp cho ALU mà chỉ cho ra tín hiệu là ALUop1 và ALUop0, từ đó điều khiển gián tiếp khối ALU.

- Các tín hiệu điều khiển của khối Control Unit:

- RegDst: lựa chọn giá trị thanh ghi đích
- MemRead: cho phép đọc giá trị vào bộ nhớ
- MemWrite: cho phép ghi giá trị vào bộ nhớ
- MemToReg: lựa chọn giá trị ghi vào thanh ghi đích
- ALUSrc: lựa chọn giá trị cho toán hạng thứ 2 của ALU
- RegWrite: cho phép ghi giá trị của 1 thanh ghi trong tập thanh ghi

- ALUop[1:0]: cho tín hiệu tương ứng với loại lệnh để truyền vào khối ALU control.

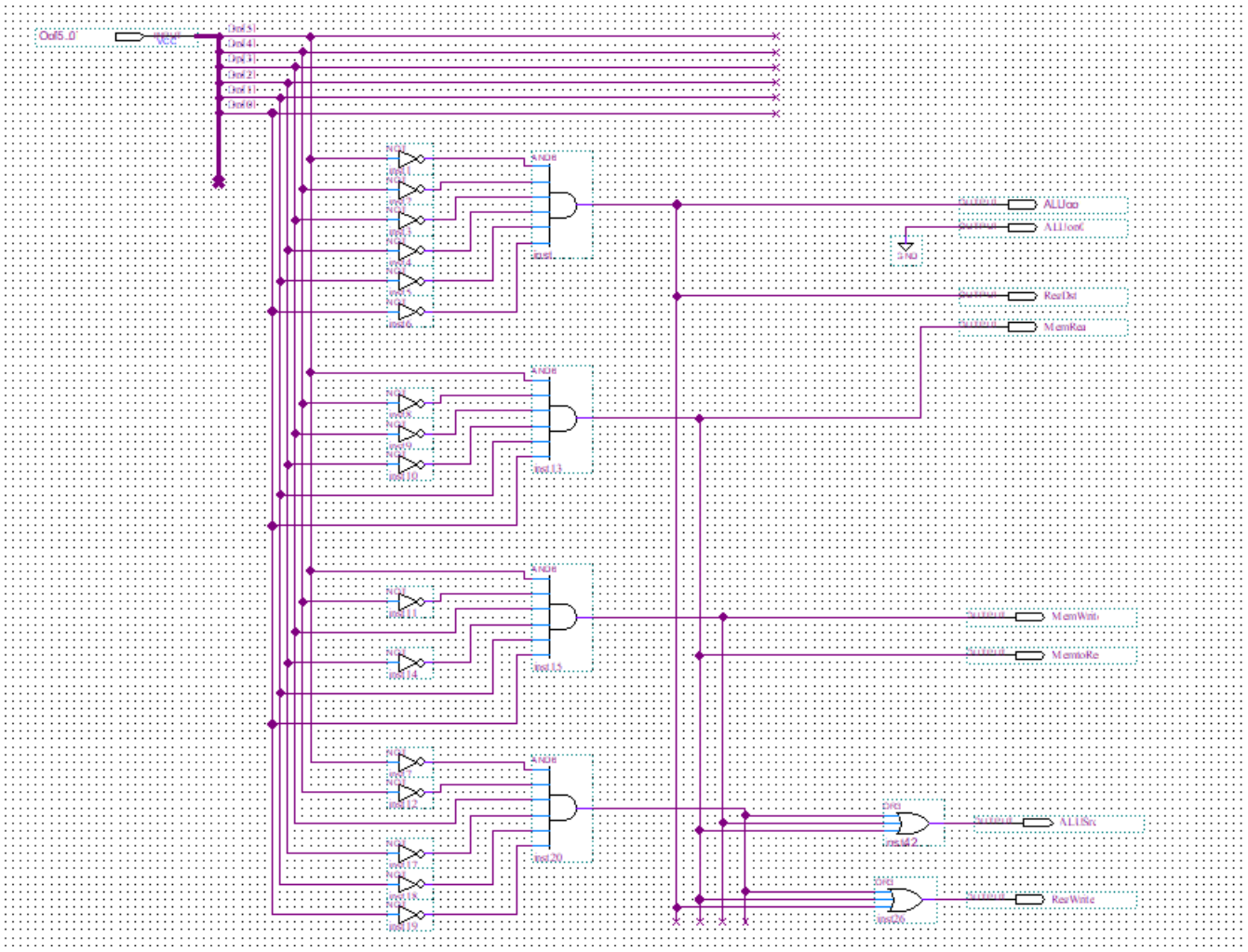
- Bảng sự thật cho tín hiệu điều khiển:

Opcode	Lệnh	ALUop[1:0]	RegDst	MemRead	MemWrite	MemToReg	ALUSrc	RegWrite
000000	R-Type	10	1	0	0	0	0	1
100011	lw	00	0	1	0	1	1	1
101011	sw	00	X	0	1	X	1	0
001000	addi	00	0	0	0	0	1	1

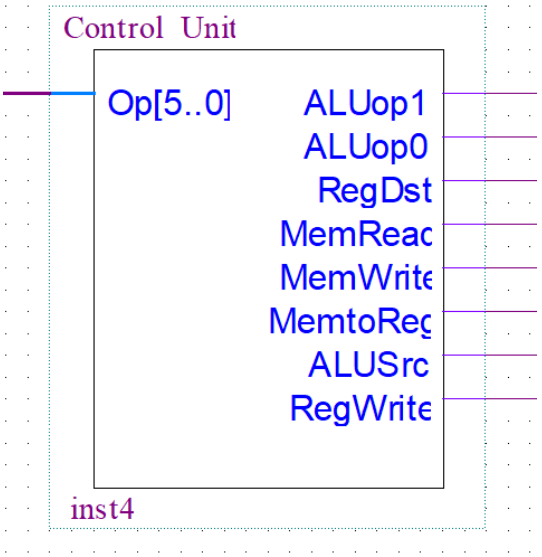
- Từ bảng sự thật trên, ta thu được các phương trình:

- ALUop1 = Op5′.Op4′.Op3′.Op2′.Op1′.Op0′
- ALUop0 = 0
- RegDst = Op5′.Op4′.Op3′.Op2′.Op1′.Op0′
- MemRead = Op5.Op4′.Op3′.Op2′.Op1.Op0
- MemWrite = Op5.Op4′.Op3.Op2′.Op1.Op0
- MemToReg = Op5.Op4′.Op3′.Op2′.Op1.Op0
- ALUSrc = Op5.Op4′.Op3′.Op2′.Op1.Op0 + Op5.Op4′.Op3.Op2′.Op1.Op0 + Op5′.Op4′.Op3.Op2′.Op1′.Op0′
- RegWrite = Op5′.Op4′.Op3′.Op2′.Op1′.Op0′ + Op5.Op4′.Op3′.Op2′.Op1.Op0 + Op5′.Op4′.Op3.Op2′.Op1′.Op0′

- Vẽ mạch Control Unit:

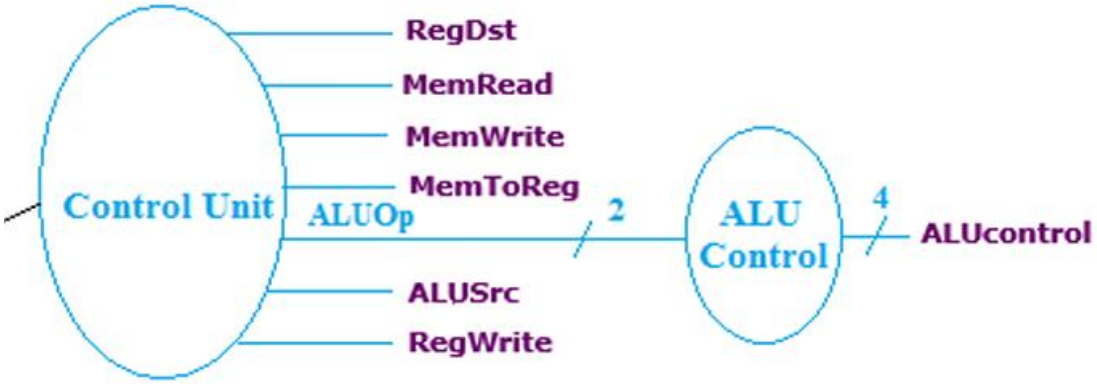


- Đóng gói khối Control Unit:



2. Thiết kế khối ALU Control

- Khối ALU Control dựa vào tín hiệu của ALUop[1:0] của khối Control Unit và dựa vào giá trị của trường Function của mã máy Inst[5:0] để xác định giá trị ALUControl[3:0] truyền vào khối ALU để lựa chọn phép tính tương ứng.



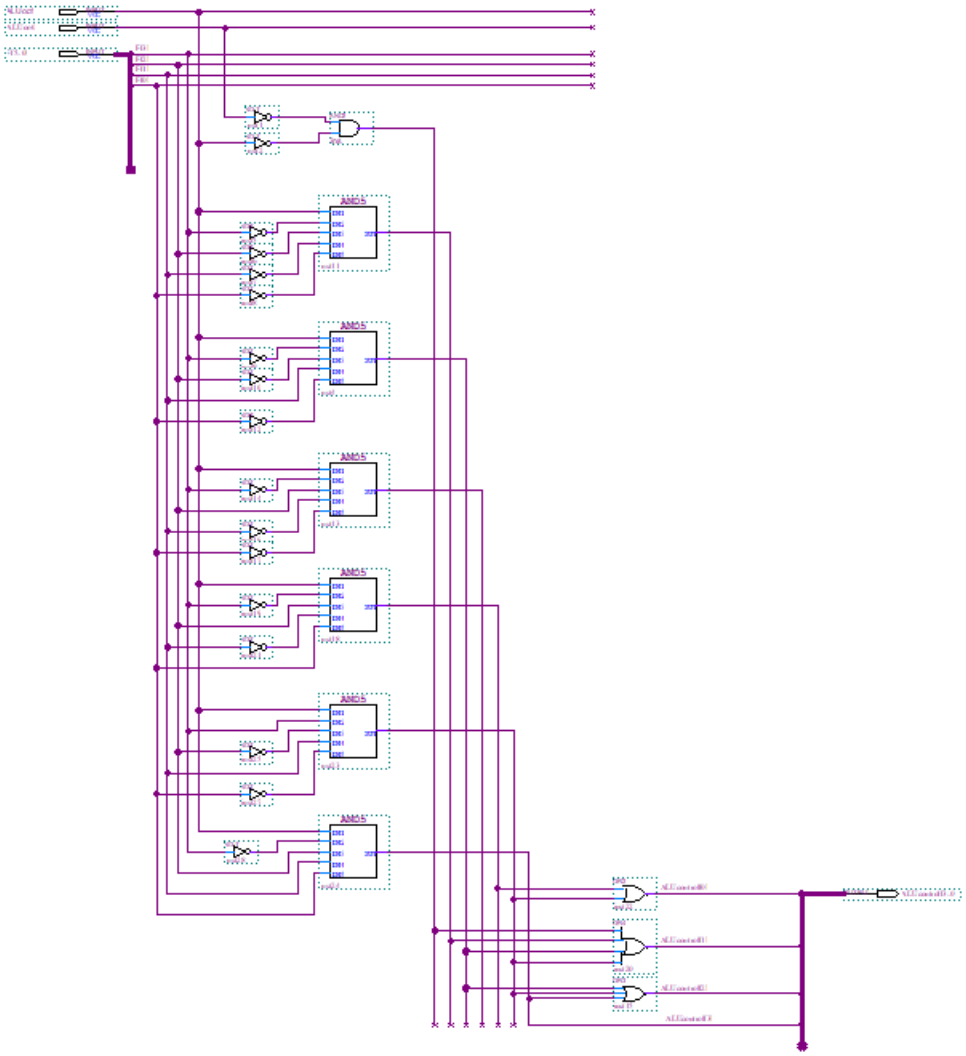
- Bảng sự thật cho tín hiệu điều khiển:

ALUop1	ALUop0	F5	F4	F3	F2	F1	F0	ALUControl	Lệnh
0	0	X	X	X	X	X	X	0010	lw, sw
1	X	X	X	0	0	0	0	0010	add
1	X	X	X	0	0	1	0	0110	sub
1	X	X	X	0	1	0	0	0000	and
1	X	X	X	0	1	0	1	0001	or
1	X	X	X	1	0	1	0	0111	slt
1	X	X	X	0	1	1	1	1100	nor

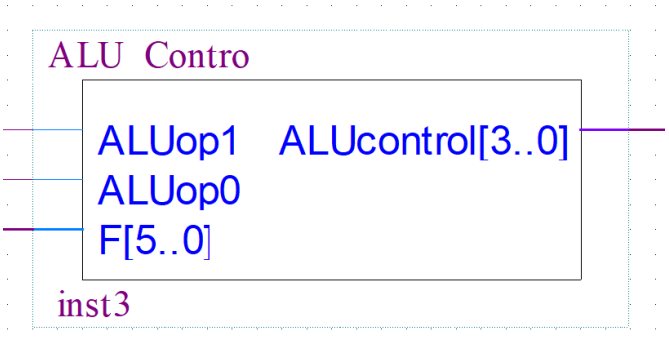
- Từ bảng sự thật trên, ta thu được các phương trình:

  - ALUControl[3] = 0
  - ALUControl[2] = ALUop1.F3'.F2'.F1.F0' + ALUop1.F3.F2'.F1.F0' = ALUop1. F2'.F1.F0'
  - ALUControl[1] = ALUop1'.ALUop0' + ALUop1.F3'.F2'.F1'.F0' + ALUop1.F3'.F2'.F1.F0' + ALUop1.F3.F2'.F1.F0' = ALUop1'.ALUop0'+ ALUop1.F3'.F2'.F0' + ALUop1.F3.F2'.F1.F0'
  - ALUControl[0] = ALUop1.F3.F2'.F1.F0' + ALUop1.F3'.F2.F1'.F0

- Vẽ mạch:



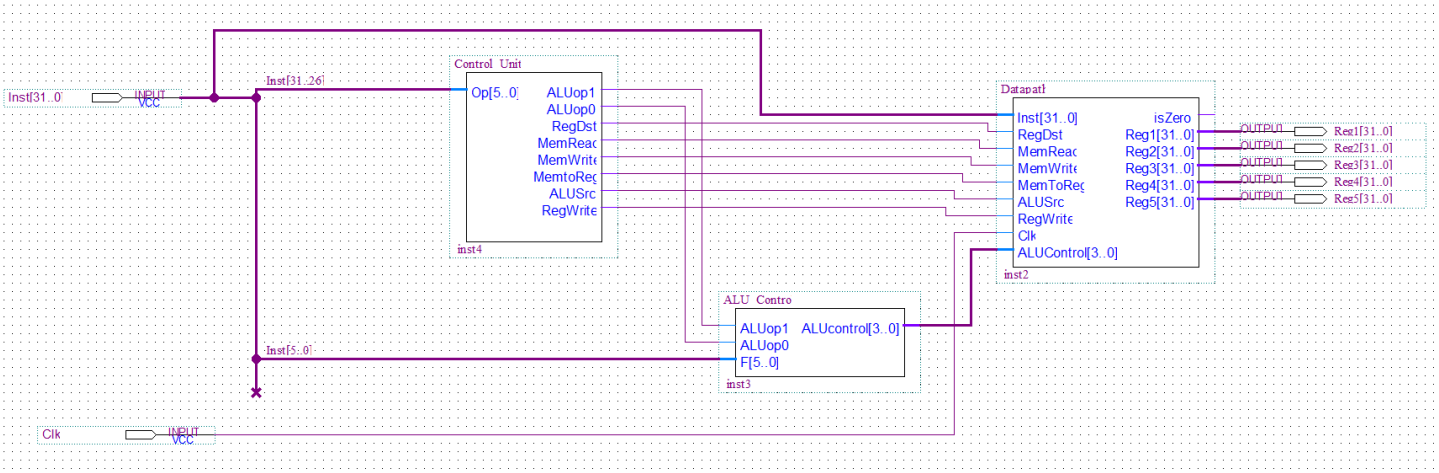
- Đóng gói khối ALU Control:



C. THIẾT KẾ KIẾN TRÚC MIPS, THỰC THI VÀ MÔ PHỎNG CÁC LỆNH MIPS

1. Hoàn thiện kiến trúc MIPS

- Sau khi đã thiết kế vào đóng gói khối Datapath và Control Unit, ta tiến hành kết nối các thành phần này lại với nhau:



Hình 5. Bộ xử lý MIPS 32 bit đơn giản

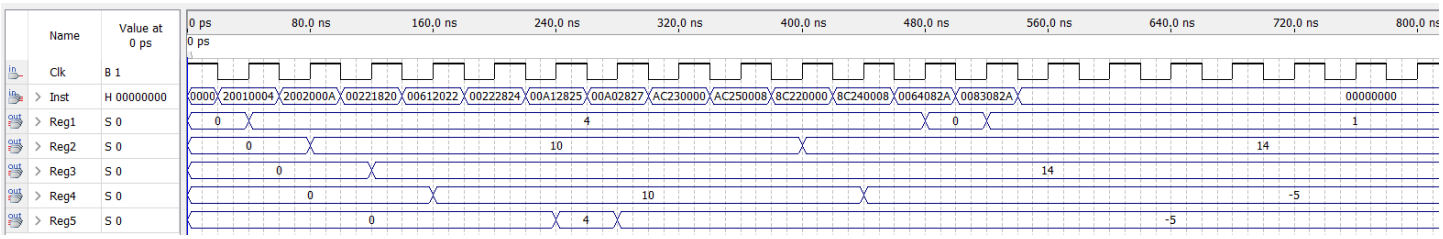
2. Thực thi và mô phỏng các lệnh MIPS

2.1. Thực thi đoạn chương trình MIPS:

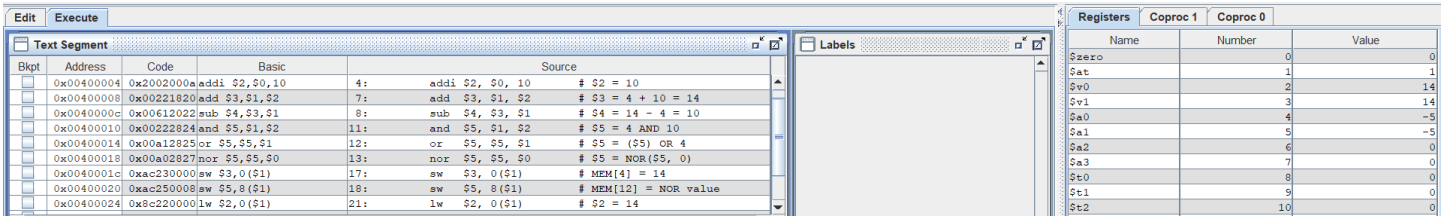
- Sử dụng đoạn code MIPS dưới đây để thực thi trên kiến trúc MIPS vừa thiết kế:

Code MIPS	Mã máy Inst[31:0] ở HEX
`# Khởi tạo giá trị và địa chỉ`	
`addi $1, $0, 4  # $1 = 4 (địa chỉ < 16)`	20010004
`addi $2, $0, 10  # $2 = 10`	2002000a
`# Test ADD, SUB`	
`add $3, $1, $2  # $3 = 4 + 10 = 14`	00221820
`sub $4, $3, $1  # $4 = 14 - 4 = 10`	00612022
`# Test AND / OR / NOR`	
`and $5, $1, $2  # $5 = 4 AND 10`	00222824
`or $5, $5, $1  # $5 = ($5) OR 4`	00a12825
`nor $5, $5, $0  # $5 = NOR($5, 0)`	00a02827
`# STORE vào Data Memory tại địa chỉ trong $1`	
`sw $3, 0($1)  # MEM[0] = 14`	ac230000
`sw $5, 8($1)  # MEM[2] = NOR value`	ac250008
`# LOAD lại từ Data Memory`	
`lw $2, 0($1)  # $2 = 14`	8c220000
`lw $4, 8($1)  # $4 = NOR value`	8c240008
`# Test SLT`	
`slt $1, $3, $4 # $1 = ($2 < $3) ? 1 : 0`	0064082a
`slt $1, $4, $3 # $1 = ($3 < $2) ? 1 : 0`	0083082a

2.2. Mô phỏng kết quả:



Hình 6. Mô phỏng kết quả trên Quartus



Hình 7. Kiểm tra trên mô phỏng Mars 4.5